

Lab 3: Architectural Style Justification

Problem Statement #42: Internal Microservice Catalog & Health Portal

NAME : ABHISHEK SRN : PES1UG24CS650

Architecture Selection

We chose Microservices Architecture for the Internal Microservice Catalog & Health Portal System.

1. Architectural Choice

The system is built as small, independently deployable services behind a single API Gateway: Microservice Catalog, Dependency Manager, API Documentation Manager, Health Monitoring, Downtime & Alert Manager and Auth Service. Each service exposes its own provided interface (for example ICatalogAPI, IHealthStatus) and uses the appropriate shared data store, as shown in the component diagram. Layered architecture was rejected because its tight coupling makes it hard to scale one part (such as the health pinger) without redeploying everything. Client-Server was rejected because a single central server is a single point of failure, which is unacceptable for a tool whose purpose is to detect failures.

2. Two Reasons for the Choice

- **Reason 1: Fault isolation for continuous monitoring.** FR-001 requires health probes every 30 seconds and FR-005 requires downtime to be recorded after 3 failed probes. With microservices, the Health Monitoring and Downtime & Alert services keep running even if the API Documentation Manager or Dependency Manager crashes or is being redeployed, so monitoring and alerting are not lost because of a fault in a less critical feature.
- **Reason 2: Independent scaling and deployment of different workloads.** The workloads in this portal are very different: Health Monitoring runs a constant timer-driven probe loop, while the Dependency Manager must serve an interactive graph of up to 200 services (NFR-001), and the Catalog and API Documentation services are mostly read-heavy. Each can be scaled, updated and tested on its own, and teams can change one service without touching the others.

3. Security Advantage

All user traffic enters through the API Gateway, which asks the Auth Service to validate the user's token on every request (ITokenValidation, backed by the Identity Provider). No internal service is directly reachable by users, so the catalog and API documentation are only returned to authenticated and authorized users, which satisfies NFR-002. Because each service has its own interface and access is controlled through the API Gateway and Auth Service, a compromise of one service does not automatically expose the data of the others.

4. Performance Benefit

Services scale independently and use separate databases. The write-heavy probe results and downtime events are stored in the

Health & Downtime database, so they do not slow down catalog and documentation queries in the Catalog & Docs database. The Dependency Manager can be given more instances alone to keep the 200-service graph rendering smoothly under peak load (NFR-001), instead of scaling the whole system.

Trade-off

Microservices add network latency and operational complexity. We limit this by using lightweight REST/JSON interfaces, routing all client requests through one gateway, and keeping the number of services small (six).